

QUE · MANUAL QA

Hands-on Tester Walkthrough

Product: **Que** (SchemaGraph) · Path: `adc/schemagraph`

Companion to: `Que-Local-Setup-Guide.pdf` (stack) + `MANUAL_TEST_PLAN.md` (checklist)

Document version: **0.8** · Updated: 30 Jul 2026

Goal: walk a new tester from empty account → 4 connectors → workspace → AI chat → jobs

Use this PDF **side-by-side** with the running app. Mark PASS / FAIL in the Result column. Do steps **one at a time** — seed data and cloud accounts are built as you go, not all at once.

Contents

1. How to use this guide
2. Pre-flight — stack must be up
3. Test journey map
4. Step 1 — Create Que account (admin / owner)
5. Step 2 — PostgreSQL (create account, SQL, commands, Que connect)
6. Step 3 — MongoDB (Docker, seed, dump, Que connect)
7. Step 4 — Databricks (fixture + optional live)
8. Step 5 — Excel + CSV pack (~1 lakh rows)
9. Step 6 — Workspace alignment & joins (HITL)
10. Step 7 — OpenAI BYOK
11. Step 8 — AI Chat → Jobs → Deploy
12. Sign-off

0. How to use this guide

Role	What you do
Tester	Follow each step in the browser / cloud console. Tick Result. Capture screenshots on FAIL.
Helper (eng / Cursor)	Builds seed scripts, Docker DBs, and Excel files <i>one step ahead</i> when you say “next”.

Connector reality: Que does *not* ship a native SQL Server connector. Supported creatable types: **PostgreSQL** · **MongoDB** · **Databricks** · **Snowflake** · **Excel** · **CSV**. For a 4-connector POC use PG + Mongo + Databricks + Excel (Snowflake optional via fixture).

Que is **schema-first**. Large row counts make demos feel real, but Que mainly syncs tables/columns/keys — not a full warehouse dump. Prefer shared keys like `customer_id` / `order_id` across sources so joins are obvious.

Field	Fill in
Environment	Local · UI <code>http://localhost:5174</code> · API <code>http://localhost:8787</code>
Build / commit	_____
Tester	_____
Date	_____
Workspace name	_____

1. Pre-flight — stack must be up

If the stack is not running, follow [docs/Que-Local-Setup-Guide.pdf](#) first, then return here.

#	Check	Expected	Result
1.1	GET <code>http://localhost:8787/health</code>	<code>{"ok":true, ... }, authDisabled: false</code>	☐ PASS ☐ FAIL
1.2	Open <code>http://localhost:5174</code>	Que login / create-account page	☐ PASS ☐ FAIL
1.3	<code>docker ps</code> shows <code>stitch-pg</code>	Status Up (port 5432)	☐ PASS ☐ FAIL
1.4	Optional: <code>stitch-mongo</code>	Up (needed for Step 3)	☐ PASS ☐ FAIL ☐ N/A

```
# PowerShell smoke
Invoke-RestMethod http://localhost:8787/health | ConvertTo-Json
docker ps --filter name=stitch --format "{{.Names}} {{.Status}}"
```

2. Test journey map

Step	What	Status in this PDF
1	Create Que account → owner (admin) of first workspace	READY detailed below
2	PostgreSQL <code>customer_demo</code> DB + seed + connect in Sources	READY seed applied; UI steps below
3	MongoDB <code>customer_demo</code> + seed + connect	READY seed + dumps; UI steps below
4	Databricks Unity fixture (+ optional live account)	READY fixture + dumps; UI steps below
5	Excel + CSV: 15 files · 109k rows	READY pack on disk; upload in UI
6	Workspace graph — alignment / promote joins	READY detailed steps below
7	Settings → OpenAI BYOK	READY detailed steps below
8	Chat → draft job → notebook / validate / deploy	READY detailed steps below

3. Step 1 — Create Que account (admin / owner)

Creating a workspace makes you **owner**. That is full admin for connectors, invites, secrets, and exports. You do *not* need a separate “assign admin” click.

3.1 Open the app

1. Browser → `http://localhost:5174`
2. If you see Sign In, click **CREATE ACCOUNT**

3.2 Fill the form

Field	Guidance	Your value
Display name	Your name	_____
Email	Any unused email shape (must include @)	_____
Password	At least 8 characters	(do not write in shared PDF)
First workspace name	e.g. <code>Que Test Lab</code>	_____

Click **CREATE ACCOUNT**.

3.3 Verify owner access

#	Step	Expected	Result
3.3.1	After create, you land in the product (Workspace or primary nav)	Logged in; no login loop	☐ PASS ☐ FAIL
3.3.2	Open Settings	Team / invite controls visible (owner)	☐ PASS ☐ FAIL
3.3.3	Open Sources	Empty or catalog; you can start Add connector	☐ PASS ☐ FAIL
3.3.4	Confirm workspace name in chrome / settings	Matches what you typed	☐ PASS ☐ FAIL

If create fails:

- `email already registered` → use another email or Sign In
- Network / CORS → API not on `:8787` or UI API base wrong
- Blank page → check Vite terminal for compile errors

Demo owner `dev@stitch.local` may still exist from seed. Prefer your **own** account for this walkthrough so the workspace starts clean.

4. Step 2 — PostgreSQL: how we created it, SQL dump, commands, Que connector

HELPER SEED READY · YOU CONNECT IN UI

Shareable SQL + README live outside the PDF at:

`adc/schemagraph/docs/tester-fixtures/postgres/`

Prefer `customer_demo_rebuild.sql` (small) · full INSERT dump: `customer_demo_data.sql` (~1.8 MB) + `customer_demo_schema.sql`.

4.0 What “Postgres account” means here

We did **not** open a cloud Postgres (RDS / Neon / Supabase) account for this local test. Credentials come from the **Docker container** that already runs Que’s metadata DB.

Thing	Value	Where you get it
Container	<code>stitch-pg</code>	<code>docker ps --filter name=stitch-pg</code>
Image (first create)	<code>pgvector/pgvector:pg16</code>	Local Setup Guide / command below
User	<code>stitch</code>	Docker env <code>POSTGRES_USER</code>
Password	<code>stitch</code>	Docker env <code>POSTGRES_PASSWORD</code>
Metadata DB (Que internal)	<code>stitch</code>	Docker env <code>POSTGRES_DB</code> — do not use as a Source
Source DB (connector target)	<code>customer_demo</code>	Created by bootstrap script (separate database on same server)
Host / port from Que’s view	<code>localhost:5432</code>	Docker publish <code>-p 5432:5432</code>

4.1 Commands — create Postgres “account” (Docker) if missing

```
# Check
docker ps -a --filter name=stitch-pg

# Start existing
docker start stitch-pg

# FIRST TIME ONLY – creates user stitch / password stitch / db stitch
docker run -d --name stitch-pg `
  -e POSTGRES_USER=stitch `
  -e POSTGRES_PASSWORD=stitch `
  -e POSTGRES_DB=stitch `
  -p 5432:5432 `
  pgvector/pgvector:pg16

# Wait until ready
docker exec stitch-pg pg_isready -U stitch -d stitch
```

That single Docker user is both the Que metadata login (`DATABASE_URL`) and the customer-source login. Only the **database name** differs (`stitch` vs `customer_demo`).

4.2 Commands — create source DB + load SQL (what we ran)

```
# From api folder
cd D:\ADC\prosols\adc\schemagraph\api

# Creates database customer_demo (if needed) + applies db/002_customer_demo.sql
npm run bootstrap:demo-source

# Replaces tiny seed with ~11k rows (db/002b_customer_demo_bulk.sql)
node scripts/applyCustomerDemoBulk.js

# Verify
docker exec -i stitch-pg psql -U stitch -d customer_demo -c "
    SELECT 'customers' t, COUNT(*) FROM customers
    UNION ALL SELECT 'products', COUNT(*) FROM products
    UNION ALL SELECT 'orders', COUNT(*) FROM orders
    UNION ALL SELECT 'order_items', COUNT(*) FROM order_items;
"
```

Alternate one-file apply (shareable rebuild SQL):

```
# Create empty DB if needed
docker exec -i stitch-pg psql -U stitch -d postgres -c "CREATE DATABASE customer_demo;"

# Apply shareable pack
Get-Content D:\ADC\prosols\adc\schemagraph\docs\tester-fixtures\postgres\customer_demo_rebuild.sql -
Raw |
    docker exec -i stitch-pg psql -U stitch -d customer_demo
```

4.3 Shareable dump files (outside PDF)

Folder: `docs/tester-fixtures/postgres/`

File	What	When to use
<code>README.md</code>	Same story as this section	Hand to another tester
<code>customer_demo_rebuild.sql</code>	DDL + <code>generate_series</code> seed (~11k)	Best to share — small, rebuilds cleanly
<code>002_customer_demo.sql</code>	Schema + 3 demo customers	Matches repo <code>db/</code>
<code>002b_customer_demo_bulk.sql</code>	Bulk truncate + 11k insert	Apply after 002
<code>customer_demo_schema.sql</code>	<code>pg_dump --schema-only</code>	Exact live DDL
<code>customer_demo_data.sql</code>	<code>pg_dump --data-only --inserts</code> (~1.8 MB)	Exact row dump if you need INSERT statements

```
# Re-export dumps yourself later
docker exec stitch-pg pg_dump -U stitch -d customer_demo --schema-only --no-owner --no-privileges
docker exec stitch-pg pg_dump -U stitch -d customer_demo --data-only --inserts --no-owner --no-privileges
```

4.4 SQL we added (compact — full text in rebuild file)

Tables & FKs:

```
customers (id PK, email UNIQUE, full_name, created_at)
products  (id PK, sku UNIQUE, name, unit_price)
orders    (id PK, customer_id → customers, status, ordered_at)
order_items (id PK, order_id → orders, product_id → products, quantity)
```

Bulk row recipe (see `customer_demo_rebuild.sql` for complete script):

```
-- 2,500 customers  (ada/grace/alan + user{N}@example.com)
-- 500 products     (SKU-0001 ...)
-- 3,500 orders      (FK customer_id)
-- 4,500 order_items (FK order_id, product_id)
-- TOTAL ≈ 11,000 rows
```

4.5 Que connector details — where each field comes from

Que Sources field	Value to type	Source of truth
Display name	postgres_customer_demo	Your choice (UI only)
Type	PostgreSQL	Connector catalog
Host	localhost	Docker published on your machine
Port	5432	docker port stitch-pg / -p 5432:5432
Database	customer_demo	Bootstrap created this DB
Schema	public	Default Postgres schema
User	stitch	POSTGRES_USER
Password	stitch	POSTGRES_PASSWORD
URL (optional reference)	postgresql://stitch:stitch@localhost:5432/customer_demo	

Never point the connector at database `stitch` — that is Que's own metadata (users, jobs, encrypted secrets). Always `customer_demo` for this test.

4.6 UI clicks (tester)

1. Open `http://localhost:5174` · signed in as your owner account
2. Nav → **Sources**
3. Add connector / catalog → **PostgreSQL**
4. Fill §4.5 · Create / Save
5. Open connection → **Sync**
6. Nav → **Workspace** → four tables on canvas

#	Step	Expected	Result
4.1	<code>stitch-pg</code> Up; seed counts = 11,000	Docker + SQL OK	☐ PASS ☐ FAIL
4.2	Create PostgreSQL connection (§4.5)	Saved; password masked	☐ PASS ☐ FAIL
4.3	Sync	customers, products, orders, order_items	☐ PASS ☐ FAIL
4.4	Workspace	Four tables; FK edges may show	☐ PASS ☐ FAIL

If sync fails: wrong password → `stitch / stitch` · missing DB → `npm run bootstrap:demo-source` · empty tables → `node scripts/applyCustomerDemoBulk.js` · Docker down → `docker start stitch-pg`

5. Step 3 — MongoDB: how we created it, dump, commands, Que connector

HELPER SEED READY · YOU CONNECT IN UI

Shareable pack outside the PDF:

`adc/schemagraph/docs/tester-fixtures/mongodb/`

Prefer `seed_rebuild.mjs` (regenerates) · JSON dumps in `json/` · BSON in `bson_dump/`.

5.0 What “Mongo account” means here

Local Docker Mongo (`mongo:7`) with **no auth** by default — not Atlas / cloud. URI is enough for Que.

Thing	Value	Where you get it
Container	<code>stitch-mongo</code>	<code>docker ps --filter name=stitch-mongo</code>
Image	<code>mongo:7</code>	First-time <code>docker run</code>
Host / port	<code>localhost:27017</code>	<code>-p 27017:27017</code>
Auth	None (local)	Default official image
URI	<code>mongodb://localhost:27017</code>	Host + port
Database	<code>customer_demo</code>	Created/filled by seed script

5.1 Commands — create Mongo container if missing

```
# Check / start
docker ps -a --filter name=stitch-mongo
docker start stitch-mongo

# FIRST TIME ONLY
docker run -d --name stitch-mongo -p 27017:27017 mongo:7

# Ping
docker exec stitch-mongo mongosh --quiet --eval "db.runCommand({ ping: 1 }).ok"
```

5.2 Commands — seed data (what we ran)

```
cd D:\ADC\prosols\adc\schemagraph\api
npm run bootstrap:demo-mongo-bulk
# same as: node scripts/applyCustomerDemoMongoBulk.js

# Verify
docker exec stitch-mongo mongosh --quiet customer_demo --eval "
  print('profiles', db.profiles.countDocuments());
  print('events', db.events.countDocuments());
  print('sessions', db.sessions.countDocuments());
"
```

Collection	Docs	Join keys vs Postgres
<code>profiles</code>	2,500	<code>email</code> , <code>pg_customer_id</code> (= PG <code>customers.id</code>)
<code>events</code>	8,000	<code>email</code> , <code>pg_customer_id</code>
<code>sessions</code>	3,000	<code>email</code> , <code>pg_customer_id</code>
Total	13,500	Same <code>user{N}@example.com</code> pattern as PG

5.3 Shareable dump files (outside PDF)

Folder: docs/tester-fixtures/mongodb/

File / folder	What	When to use
README.md	Same story	Hand to another tester
seed_rebuild.mjs	Regenerates all docs	Best small share
json/*.json	mongoexport --jsonArray	Readable / import
bson_dump/	mongodump	Fast restore via mongorestore

```
# Re-export later
docker exec stitch-mongo mongodump --db=customer_demo --out=/tmp/que_mongo_dump
docker exec stitch-mongo mongoexport --db=customer_demo --collection=events --jsonArray --out=/tmp/events.json

# Restore BSON
docker cp docs\tester-fixtures\mongodb\bson_dump stitch-mongo:/tmp/customer_demo_restore
docker exec stitch-mongo mongorestore --drop --db=customer_demo /tmp/customer_demo_restore
```

5.4 Document shape (schema Que will infer)

```
profiles:  _id, customer_key, email, full_name, tier, pg_customer_id,
           prefs.marketing, prefs.locale, created_at
events:    _id, user_id, email, pg_customer_id, event_type, severity,
           meta.browser, meta.ip, payload.sku?, payload.amount?, status_hint, created_at
sessions:  _id, user_id, email, pg_customer_id, session_token,
           device.os, device.app_version, started_at, duration_sec
```

Que probes sample docs and flattens nested fields (dot paths). It does **not** import all 13.5k documents into the graph — schema + capped samples only.

5.5 Que connector details — where each field comes from

Que Sources field	Value	Source of truth
Display name	mongo_customer_demo	Your choice
Type	MongoDB	Connector catalog
URI	mongodb://localhost:27017	Docker host + port (no auth)
Database	customer_demo	Seed script DB name

5.6 UI clicks (tester)

1. Nav → **Sources** → Add → **MongoDB**
2. URI `mongodb://localhost:27017` · Database `customer_demo`
3. Create / Save → **Sync**
4. Workspace → expect `profiles`, `events`, `sessions` (with PG tables still there)

#	Step	Expected	Result
5.1	<code>stitch-mongo</code> Up; counts 2500 / 8000 / 3000	Docker + seed OK	☐ PASS ☐ FAIL
5.2	Create MongoDB connection (\$5.5)	Saved	☐ PASS ☐ FAIL
5.3	Sync	Three collections as tables; nested fields flattened	☐ PASS ☐ FAIL
5.4	Workspace	Mongo + Postgres tables visible together	☐ PASS ☐ FAIL

If sync fails: Docker down → `docker start stitch-mongo` · empty DB → `npm run bootstrap:demo-mongo-bulk` · wrong database name → must be `customer_demo` · connection refused → confirm port `27017` .

6. Step 4 — Databricks: fixture account path, dump, commands, Que connector

FIXTURE READY

YOU CONNECT IN UI

Shareable pack: `adc/schemagraph/docs/tester-fixtures/databricks/`

Runtime fixture (API): `api/fixtures/databricks_unity_demo.json`

No cloud account required for fixture mode — optional live steps below.

6.0 What “Databricks account” means here

Mode	Account?	What you provide
Fixture (this POC)	None	Path to JSON under <code>api/fixtures/</code>
Live (optional later)	Databricks workspace + SQL warehouse + PAT	Host, Warehouse ID, Token — see <code>WHERE_TO_GET_LIVE_CREDS.md</code>

Fixture is schema + sample values only (like a Unity Catalog snapshot). Que does not spin up a lakehouse or load lakhs of rows for this connector.

6.1 Commands — verify / refresh fixture

```
# Confirm fixture loads
cd D:\ADC\prosols\adc\schemagraph\api
node --input-type=module -e "import { readFileSync } from 'fs'; const
j=JSON.parse(readFileSync('fixtures/databricks_unity_demo.json','utf8'));
console.log(j.catalog+'.'+j.schema); console.log(j.tables.map(t=>t.name+'
'+t.entityKind+'').join('\n'))"

# Re-copy into share folder after edits
Copy-Item fixtures\databricks_unity_demo.json ..\docs\tester-fixtures\databricks\ -Force
```

Object	Kind	Cross-source keys
<code>dim_customer</code>	TABLE	<code>email</code> , <code>pg_customer_id</code>
<code>fact_orders</code>	TABLE	FK <code>customer_sk</code> → <code>dim_customer</code>
<code>gold_campaign_attribution</code>	VIEW	<code>email</code>

Catalog / schema in fixture: `main.analytics`

6.2 Shareable dump files (outside PDF)

File	What
<code>README.md</code>	Full recreate + Que fields
<code>databricks_unity_demo.json</code>	Primary dump — schema + samples + FKs
<code>sample_rows_dim_customer.csv</code>	Human-readable sample rows
<code>sample_rows_fact_orders.csv</code>	Human-readable sample rows
<code>WHERE_TO_GET_LIVE_CREDS.md</code>	Host / warehouse / PAT from Databricks UI

6.3 Que connector — fixture (do this now)

Que field	Value	Where from
Display name	<code>databricks_lakehouse_demo</code>	Your choice
Type	Databricks	Catalog
Auth mode	One-click fixture	Toggle on form
Fixtures path	<code>fixtures/databricks_unity_demo.json</code>	Relative to <code>api/</code> (default)
Catalog	<code>main</code>	Matches JSON (default)
Schema	<code>analytics</code>	Matches JSON (default)

1. Sources → Add → **Databricks**
2. Leave mode on **One-click fixture**
3. Keep default fixtures path · Catalog `main` · Schema `analytics`
4. Create → **Sync**
5. Workspace → three lakehouse objects appear with PG + Mongo

6.4 Optional — live Databricks (skip unless you have a workspace)

Field	Where to get it
Host	Workspace URL hostname (no https) — e.g. <code>adb-....azuredatabricks.net</code>
Warehouse ID	SQL → SQL Warehouses → open warehouse → ID
Token	User Settings → Developer → Access tokens → Generate (<code>dapi...</code>)
Catalog / Schema	Unity Catalog objects that hold your tables

Details: `docs/tester-fixtures/databricks/WHERE_TO_GET_LIVE_CREDS.md`

#	Step	Expected	Result
6.1	Fixture file present under <code>api/fixtures/</code>	JSON parses; 3 tables listed	☐ PASS ☐ FAIL
6.2	Create Databricks fixture connection + Sync	<code>dim_customer</code> , <code>fact_orders</code> , <code>gold_campaign_attribution</code>	☐ PASS ☐ FAIL
6.3	Workspace shows DBX + PG + Mongo	Multi-source canvas	☐ PASS ☐ FAIL
6.4	Optional live sync	Live schema or N/A	☐ PASS ☐ FAIL ☐ N/A

If sync fails: path typo → must be `fixtures/databricks_unity_demo.json` relative to API cwd · API not running · wrong mode (live without token) → switch back to fixture · catalog/schema mismatch → use `main` / `analytics` .

7. Step 5 — Excel + CSV pack (~1 lakh rows)

PACK GENERATED · YOU UPLOAD IN UI

Shareable folder: `adc/schemagraph/docs/tester-fixtures/excel/`

15 files · 109,000 rows (9 Excel + 6 CSV) · inventory in `MANIFEST.json` + `README.md`.

7.0 What “Excel/CSV account” means

There is **no** cloud account. Files are generated on disk; you **upload** them into Que Sources (Excel and/or CSV connectors).

7.1 Commands — how we generated the pack

```
cd D:\ADC\prosols\adc\schemagraph\api
npm run fixtures:tester-excel
# same as: node scripts/generateTesterExcelPack.js

# Output
# D:\ADC\prosols\adc\schemagraph\docs\tester-fixtures\excel\
```

Optional zip to share:

```
Compress-Archive -Path D:\ADC\prosols\adc\schemagraph\docs\tester-fixtures\excel\* `
-DestinationPath D:\ADC\prosols\adc\schemagraph\docs\tester-fixtures\excel_pack_109k.zip -Force
```

7.2 File inventory (dump outside PDF)

#	File	Rows	Cols	Type
1	01_campaigns.xlsx	8,000	6	Excel
2	02_leads.xlsx	10,000	7	Excel
3	03_accounts.xlsx	8,000	7	Excel
4	04_support_tickets.xlsx	12,000	10	Excel
5	05_web_events.xlsx	15,000	8	Excel
6	06_invoices.xlsx	7,000	9	Excel
7	07_products_catalog.xlsx	5,000	12	Excel
8	08_nps_responses.csv	6,000	5	CSV
9	09_marketing_touch.csv	10,000	6	CSV
10	10_refunds.csv	4,000	8	CSV
11	11_referrals.csv	5,000	4	CSV
12	12_subscriptions.xlsx	6,000	11	Excel
13	13_device_registry.csv	3,000	15	CSV
14	14_geo_ip_lookup.csv	2,000	3	CSV
15	15_order_shipments.xlsx	8,000	14	Excel
TOTAL		109,000	3–15	15 files

Join keys: `email`, `owner_email`, `customer_id` (same domain as PG / Mongo).

7.3 Que connector — upload steps

1. Open folder `docs/tester-fixtures/excel/` in Explorer
2. **Excel batch:** Sources → Add → **Excel** → name `excel_tester_pack` → upload `01_*.xlsx ... 07_*.xlsx`, `12_*.xlsx`, `15_*.xlsx` → Save → Sync
3. **CSV batch:** Sources → Add → **CSV** → name `csv_tester_pack` → upload `08-11`, `13`, `14` → Save → Sync
4. Workspace → spreadsheet tables appear alongside PG / Mongo / Databricks

Que stores uploads under `api/uploads/{workspaceId}/...` and introspects schema + capped samples — it does not keep all 109k rows in the graph.

#	Step	Expected	Result
7.1	Pack on disk (15 files / 109k rows)	<code>MANIFEST.json</code> matches table above	☐ PASS ☐ FAIL
7.2	Upload Excel pack + Sync	Sheets become tables	☐ PASS ☐ FAIL
7.3	Upload CSV pack + Sync	CSV tables appear	☐ PASS ☐ FAIL
7.4	Workspace shows file sources with DBs	Multi-source canvas	☐ PASS ☐ FAIL

If upload fails: API down · file too large for browser timeout (upload fewer files per batch) · wrong type (put CSV on CSV connector, XLSX on Excel) · disk full under `api/uploads` .

8. Step 6 — Workspace alignment & joins (HITL)

DO THIS AFTER ALL CONNECTORS SYNC

Goal: see tables from **Postgres + Mongo + Databricks + Excel/CSV** on one canvas, review suggested joins on shared keys (`email` , `customer_id` / `pg_customer_id`), then **Promote** good joins and **Reject** noise. Que never auto-accepts joins.

8.1 Preconditions

- All Step 2–5 connectors created and **Synced**
- You are signed in as workspace **owner**
- UI `http://localhost:5174` · API `:8787` healthy

8.2 Open Workspace & check alignment

1. Nav → **Workspace**
 2. Confirm table nodes from multiple sources (PG `customers` / `orders` , Mongo `events` / `profiles` , DBX `dim_customer` / `fact_orders` , Excel/CSV sheets)
 3. Drag tables to arrange; reload page — layout should persist
 4. Optional: run stitch / join-inference from UI if a “Suggest joins” / Stitch session control is shown
- Expected join signals: same emails (`ada@example.com` , `user{N}@example.com`) and `customer_id` / `pg_customer_id` across PG ↔ Mongo ↔ Excel ↔ Databricks samples.

8.3 HITL — Promote & Reject

1. Click a **suggested** relationship edge (dashed / “suggested” styling)
2. Read the evidence (column names, samples)
3. Click **Promote** on at least one good cross-source join (e.g. PG `customers.email` ↔ Mongo `profiles.email`)
4. Click **Reject** on at least one weak/noise suggestion
5. Confirm promoted edges look “accepted / explicit”; rejected ones no longer drive export

#	Step	Expected	Result
8.1	Workspace shows multi-source tables	PG + Mongo + DBX + files visible	☐ PASS ☐ FAIL
8.2	Suggested joins appear on shared keys	At least some suggested edges	☐ PASS ☐ FAIL
8.3	Promote ≥1 join	Status accepted; evidence retained	☐ PASS ☐ FAIL
8.4	Reject ≥1 join	Rejected; not used for export	☐ PASS ☐ FAIL
8.5	Reload Workspace	Layout + review decisions persist	☐ PASS ☐ FAIL

If no suggestions: re-Sync sources · confirm shared emails exist · try Stitch session selecting two sources · check Settings that join inference is enabled.

9. Step 7 — OpenAI key (BYOK)

AFTER WORKSPACE JOINS

Bring Your Own Key — paste an OpenAI API key into **Settings**. Que stores it encrypted as a workspace secret (UI shows masked). Never commit keys to git.

9.1 Where to get the key

1. Open <https://platform.openai.com/api-keys>
2. Create a secret key · copy once (starts with `sk-`)

9.2 Save in Que

1. Nav → **Settings**
2. Expand policy / BYOK / AI section if collapsed (“Show policy, BYOK & AI settings”)
3. Find **BYOK_LLM_KEYS** / OpenAI API key field
4. Paste key → Save / Update
5. Confirm status shows OpenAI configured (workspace BYOK) — value masked as dots
6. Optional: enable **Prefer LLM chat** if shown

9.3 Verify

```
# Optional API check (with your session cookie / token)
# GET http://localhost:8787/workspaces/{workspaceId}/ai/status
# Expect openai configured / not heuristic-only
```

Or open **Chat** — composer / status should indicate LLM available (not only offline skills).

#	Step	Expected	Result
9.1	Paste OpenAI key in Settings → save	Saved; masked; toast success	☐ PASS ☐ FAIL
9.2	AI / Chat acknowledges LLM	Not “heuristic only”	☐ PASS ☐ FAIL
9.3	Key not visible in plaintext in UI after save	Dots / hint only	☐ PASS ☐ FAIL

Do not paste keys into the PDF, screenshots, or git. If a key leaks, revoke it in OpenAI and create a new one.

10. Step 8 — AI Chat → Jobs → Deploy

FINAL PRODUCT PATH

Wedge path: Chat (schema-only) → draft job → notebook → Run Test / Validate → Mark Ready → Deploy / attested export. Joins you **Promoted** in Step 6 should be preferred; AI still does not auto-accept joins.

10.1 Chat smoke

1. Nav → **Chat**
2. Try: `list tables`
3. Try: `describe customers` (or `@customers` mention)
4. Try: ask how to join Postgres customers to Mongo events / Excel leads on email
5. Confirm answers cite schema — not full warehouse dumps

10.2 Draft a job from Chat

1. Ask Chat to **draft a job** stitching two sources (e.g. PG customers → Mongo events or Excel leads)
2. Save / open the draft when offered
3. Nav → **Jobs** · confirm the job appears in the list

10.3 Jobs notebook · Results · Deploy

1. Open the job → tab **Notebook**
2. Edit SQL / markdown cells if needed · Commit / save changes
3. Click **Run Test** (dry-run) — Process/Output populate; no warehouse write
4. Optional: run mode **Validate** · **≤20** against live PG (capped rows only)
5. Open tab **Results** — preview samples from last run
6. Mark job **Ready** when satisfied
7. Open tab **Deploy** — review governance · export JSON / SQL / dbt (or dbt PR if GitHub configured)
8. Confirm attestation / signature present on export, or a clear block reason (e.g. unreviewed joins, drift)

Job URL shape: `/jobs?job={id}&tab=notebook|results|deploy`

#	Step	Expected	Result
10.1	Chat list / describe tables	Schema-only answer + citations	☐ PASS ☐ FAIL
10.2	Draft cross-source job from Chat	Job opens in Jobs list	☐ PASS ☐ FAIL
10.3	Run Test (dry-run)	Preview/output; no write	☐ PASS ☐ FAIL
10.4	Validate ≤20 (optional)	Capped live rows only	☐ PASS ☐ FAIL ☐ SKIP
10.5	Mark Ready → Deploy / export	Attested artifact or clear block	☐ PASS ☐ FAIL

Common blocks: export blocked until joins Promoted · open column drift · viewer role lacks write · LLM key missing (Chat falls back to heuristics) · live Validate needs PG/DBX credentials.

11. Sign-off

Role	Name	Sign	Date
Tester			
Eng helper			
Ready for deeper POC? (Y/N)			

Issues found

- 1. _____
- 2. _____
- 3. _____

Related docs: [Que-Local-Setup-Guide.pdf](#) · [MANUAL_TEST_PLAN.md](#) · [Que-Feature-Guide.pdf](#) . This walkthrough is versioned as HTML + PDF under [adc/schemagraph/docs/](#) and updated as each connector step is executed.